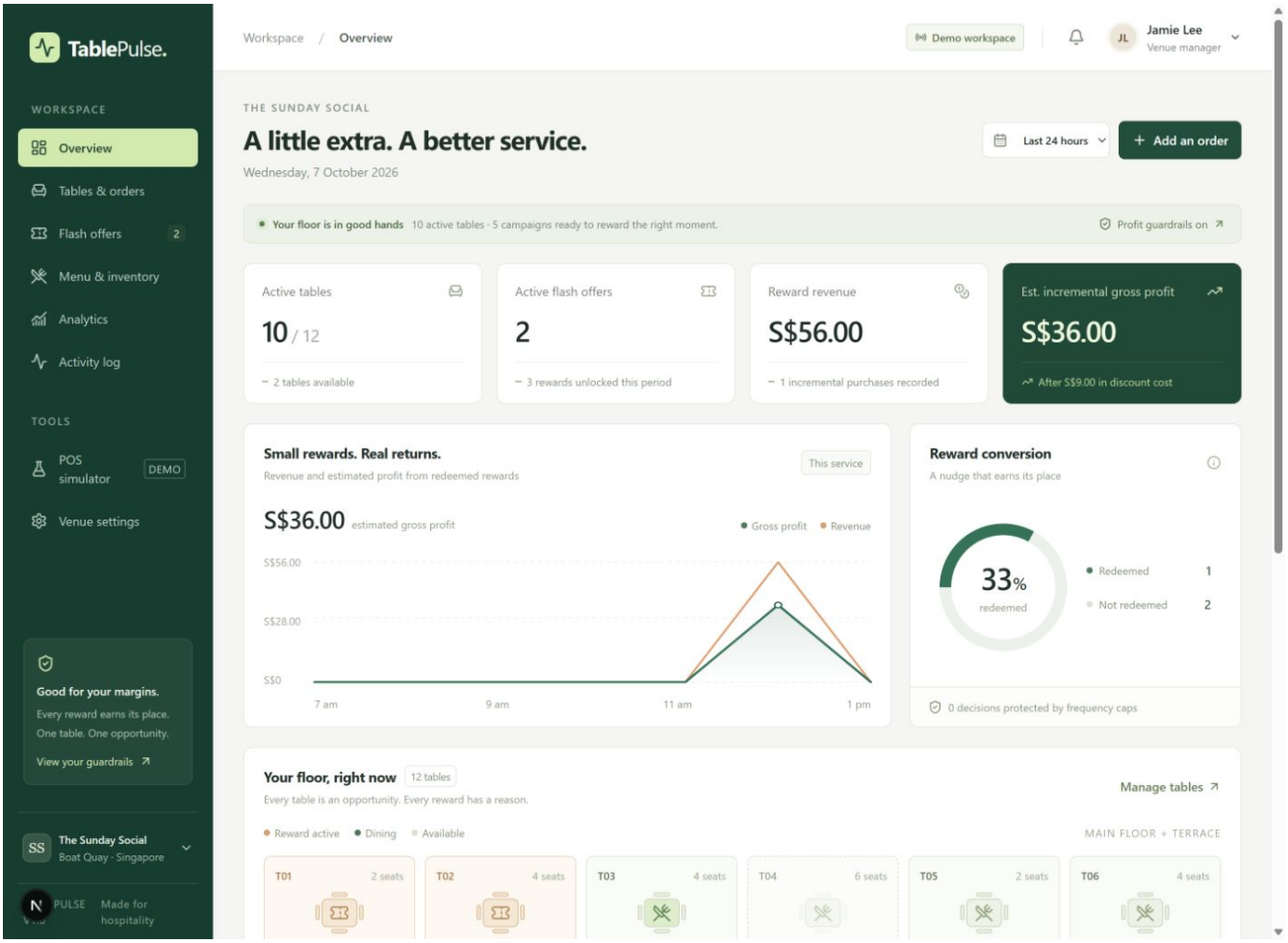
01 / PRODUCT, BACKEND AND TABLE DEVICES

# TablePulse

# Small table rewards. A measurable commercial purpose.

A practical guide for venue owners, managers and the team implementing the system. It explains what the MVP does, how its backend works, and how to connect a screen or LED device at each table.

**The goal is additional profitable purchases after an organic order. A reward is occasional, capped and specific to the current table visit.**




Current implementation: Next.js + TypeScript, Supabase production repository, local demo storage, rules-based decisions and mock POS events. No payments. No live POS, Supabase project or physical hardware is connected.

Prepared 7 October 2026 | Singapore demo: The Sunday Social | Implementation guide, not a hardware vendor specification

02 / BENEFITS AND ECONOMICS

# What the venue gains

TablePulse gives staff a controlled way to suggest an extra snack, pairing or side at a relevant moment. It can reduce manual promotion decisions and makes the economics visible to managers. These are intended benefits; a pilot must establish whether actual profit increases.

| Benefit | How it works |
|---|---|
| More profitable add-ons | Offer targets a purchase absent from the current basket, such as a snack after beer or a zero-alcohol pairing after mains. |
| Less blanket discounting | The original qualifying order stays at its original price. Campaigns have margin floors, stock checks and daily allocation caps. |
| More consistent staff decisions | The backend applies the same eligibility checks and records why a reward was shown or withheld. |
| Clearer campaign performance | Managers see reward revenue, estimated gross profit, discount cost, conversion and comparisons with control tables. |
| A private guest experience | The table screen shows an offer without displaying a customer name or purchase history. |

## A worked example from the seeded menu

A table orders two Tiger draughts organically. It may then receive Korean corn chips for S$3.90. The usual chips price is S$8.00 and seeded item cost is S$1.20.

| Measure | Calculation |
|---|---|
| Reward purchase revenue | S$3.90 |
| Estimated incremental gross profit | S$3.90 - S$1.20 = **S$2.70** |
| Gross margin on reward sale | S$2.70 / S$3.90 = **69.2%** |
| Discount against list price | S$8.00 - S$3.90 = **S$4.10** |

The demo minimum profit floor is S$2.50, so S$2.70 passes that check. Stock, caps, basket relevance, safety, score and random allocation still need to pass. Waiting does not create eligibility.

**A profitable reward sale is not proof of causal lift. If the guest would have bought chips at S$8 anyway, the offer reduces gross profit on that purchase by S$4.10.**

The MVP estimates item-level gross profit. It does not model GST, service charges, labour, waste, hardware costs or full operating profit. Compare treatment and control tables before claiming uplift.

03 / OPERATIONS AND SAFEGUARDS

# The guest and staff journey

| Step | What happens |
|---|---|
| 1. Open the visit | Staff opens a table session. The display shows branding, menu highlights and optional rewards enrolment. |
| 2. Receive a normal order | The POS adapter or staff simulator submits an organic order for that session. The engine evaluates once for that new event. |
| 3. Decide whether to offer | Hard rules run first, then relevance, profit, inventory, frequency, priority and randomness contribute to the score. |
| 4. Show the reward | An eligible table sees exact items, prices, countdown and a six-character redemption code. No guest name is shown. |
| 5. Redeem through staff | Staff checks the current table and bill. The server rechecks eligibility, consumes stock and records the purchase. |
| 6. Close the visit | Closing the bill revokes the display session. The reward cannot move to the next party. |

## The commercial controls

• At most one offer per table session, including after cancellation or expiry. No automatic replacement reward.

• At most one subsidised reward per identified customer in 30 days, and no repeat of the same promotion within 60 days.

• Current table and bill only. No stacking with existing discounts. Original qualifying items must remain on the bill.

• Only in-stock, promotion-eligible items above the configured profit and score floors. Daily allocation is real, including cancelled or expired allocations.

• Alcohol offers require verified adult eligibility and no intoxication flag. Staff can block, cancel or request a safe override; overrides retain hard safeguards.

• Every evaluated decision is logged. Staff redemption failures are audited and repeated failed attempts temporarily block redemption.

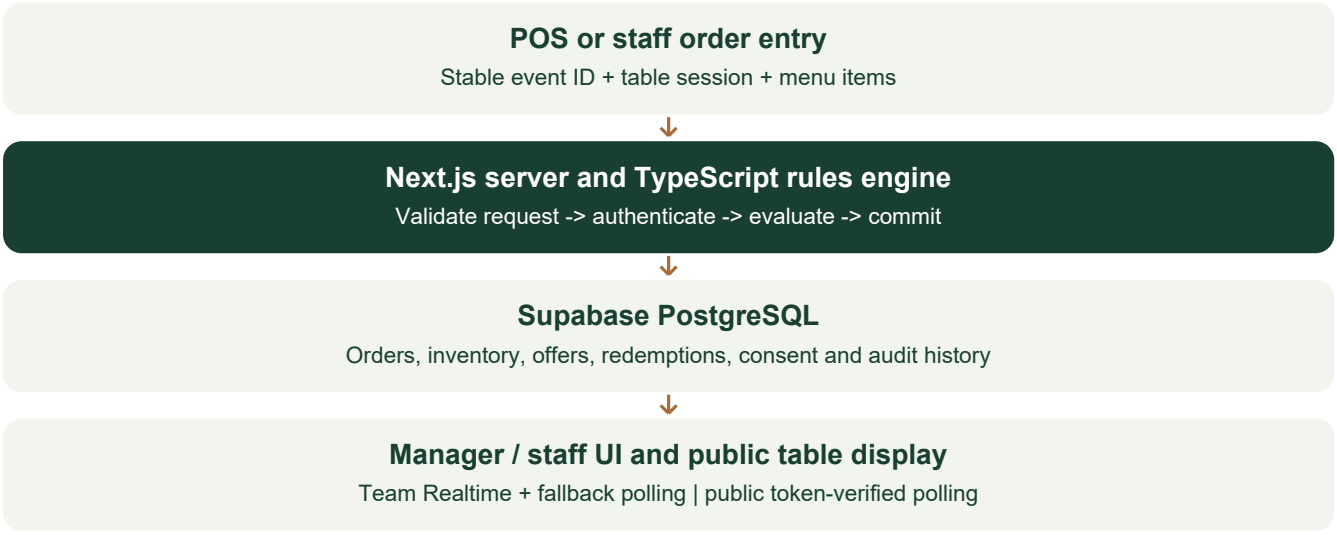
## Identification and consent are different

In live mode a subsidised reward requires a verified, stable POS customer reference. Staff or the POS links it to the visit. The reference is pseudonymized with a server-side secret. Anonymous QR enrolment records consent; it does not prove identity or recognise someone on a later visit.

Screenshots cannot be prevented physically. A screenshot or code alone is insufficient: redemption also requires authenticated staff, the original active session and server-side checks. Party-level V1 identifies one reward recipient, not every guest.

04 / APPLICATION ARCHITECTURE

# What runs in the backend




| Component | Responsibility |
|---|---|
| Next.js on Node.js | Hosts the website and API routes. The backend runs on a hosted server; it does not need the developer laptop once deployed. |
| TypeScript rules engine | Makes offer decisions in lib/engine.ts. No AI model is needed for scoring or redemption. |
| Supabase PostgreSQL | Production relational database. SQL functions apply changes atomically. The application uses the Supabase SDK, without Prisma. |
| Supabase Auth and Realtime | Team login plus authorised dashboard updates. Public displays do not get direct anonymous database access. |
| Zod validation | Checks incoming payloads, quantities, IDs and configuration values before commands execute. |
| n8n, if configured | Routes events or sends summaries and alerts. It does not own the database or decide offers. |

## Current demo versus production

The running demo persists changes in **.demo/state.json**, with a local queue and atomic file replacement. That is suitable for one demo process. Live deployments must use the Supabase repository with TABLEPULSE_DEMO=false and independently generated server secrets.

V1 selects one venue per deployment. Platform admins can manage that venue; a multi-venue fleet UI and self-service staff invitations remain future work.

05 / DATA AND SECURITY

# The database and trust boundaries

| Data group | Tables and purpose |
| --- | --- |
| Venue and team | venues, venue_users, tables, table_sessions: venue identity, team roles, physical table and current visit. |
| Customer and consent | customers, customer_consents, customer_reward_history: pseudonymous customer, preferences, consent and reward caps. |
| Sales and menu | orders, order_items, menu_items, menu_item_costs: organic bill lines, menu prices, item costs and derived margin. |
| Campaigns and decisions | offers, offer_rules, offer_eligibility_logs, offer_instances: configuration, reasons, actual table-bound reward and expiry. |
| Fulfilment and stock | offer_redemptions, inventory_availability: one-use redemption, purchase economics and remaining stock. |
| Audit trail | audit_logs: staff changes, rejected redemptions and operational decisions. |

## Consistency when requests arrive together

Venue locks and revision checks coordinate writes. Unique constraints defend one open visit per table, one offer instance per visit, one redemption per offer and unique external event IDs. Database transactions commit all effects together or roll them back. Customer-level locks and history checks protect frequency caps.

## Who can access what

Server commands authenticate the user and check venue membership and role on every mutation. PostgreSQL Row Level Security further restricts team reads. Customer profiles, consent and history are manager-only. Privileged writes use a server-only credential; it must never be placed on a table device. Supabase documents this boundary in [1].

The public API verifies a signed, expiring table token and returns only branding, table label, menu highlights and reward details. Tokens expire at most 12 hours after visit start and become invalid when the session closes. Treat display URLs as bearer credentials and avoid recording them in public logs.

**The device is a viewer. It does not decide prices, eligibility, inventory or redemption. The server remains authoritative even if the device displays an old frame.**

## Scale boundary

The MVP reconstructs a venue snapshot and commits its changes atomically. This simplifies consistency, but serializes writes within a venue. Larger deployments should benchmark it and evolve toward targeted transactions and queue-based integration retries.

06 / SCREENS VERSUS LED LIGHTS

# Connecting a device at each table

The phrase "mini LED" can describe several different products. An LCD tablet with an LED backlight can render a web page; an RGB lamp cannot. Integration depends on the exact controller, operating system and supported API.

| Device | What guests see | Integration |
|---|---|---|
| 7-10 inch tablet or browser-capable screen | Full offer, price, terms, countdown and QR/code. | Open the existing /display/[token] page in a locked-down browser. Best fit for a first pilot. |
| Small HDMI screen with a computer | The same full TablePulse display. | A Raspberry Pi or mini PC runs a browser in kiosk mode. Raspberry Pi documents this approach [2]. |
| LED matrix / scrolling text sign | Short text and code; legibility and QR support depend on resolution. | Needs compatible player software, controller firmware or a vendor API. A generic LED panel does not run Next.js. |
| RGB lamp or LED strip | A subtle status cue, such as reward available. | Needs a controller and gateway. Guests still need a screen, phone page or staff explanation for exact terms. |

## Suggested first pilot: browser screens

Use browser-capable screens on venue Wi-Fi with reliable power, secure mounts and a dedicated kiosk profile. Check viewing angle, brightness, QR readability, cleaning requirements and safe cable routing in the actual venue. This is a suggested architecture, not a certification of any untested model.

## If you mean decorative lights

Add a lamp as a secondary cue. A controller compatible with WLED can accept JSON commands for light state, brightness and colour [3]. TablePulse would need a gateway that translates reward state into those commands; this integration is not implemented in the MVP.

**A simple LED light cannot explain an offer or enforce redemption. Keep the exact price, expiry and restrictions on a readable screen or staff workflow.**

Hardware selection remains open: obtain the device model, screen size/resolution, power requirements, connectivity and controller API before buying a fleet. Prototype one physical unit first.

07 / DEVICE SESSIONS AND AGENTS

# How the right table gets the right offer

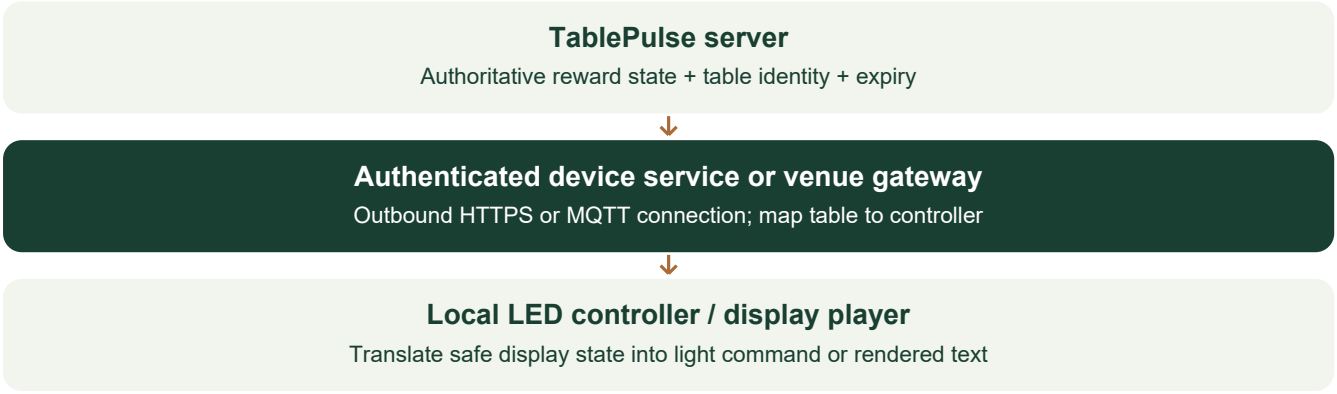
## A browser screen: the simplest connection

Each physical device is associated with a table. When staff opens a new visit, the backend issues a fresh signed display URL for that visit. Staff opens that URL on the device. The existing screen polls /api/display every three seconds and renders the server-approved state.

An active reward clears after expiry. Closing the bill invalidates the token; the current page then shows an end-of-visit message. It does not automatically attach itself to the next party. For an unattended installation, add secure device provisioning, table assignment and automatic session handover.

| Meaning of "agent" | Do we need it? |
|---|---|
| AI agent / language model | **No.** V1 uses explicit business rules. An AI model is not needed to decide margins, caps or redemption. |
| Backend web service | **Yes.** The deployed Next.js service handles requests and rules; Supabase stores data and provides Auth/Realtime. |
| Device kiosk / management software | **Useful.** Keeps the browser full-screen, starts it after reboot and supports device updates. It does not decide rewards. |
| Local hardware gateway | **Only for certain hardware.** Needed when a cloud server cannot directly control a lamp, matrix sign or local POS. |
| Background worker / n8n | **Optional.** Useful for retry queues, device monitoring, scheduled summaries or alerts. |

## Proposed lamp or matrix architecture - future work




MQTT is a publish/subscribe transport; ESP-MQTT supports it and documents TLS examples [4]. It is an optional transport design, not part of the current TablePulse backend. Do not expose local LED controllers through public router ports.

08 / PRACTICAL HARDWARE OPERATION

# Device reliability and security

The production device layer should handle device identity separately from customer identity. One table can have many visits; the device persists, while each visit token is short-lived.

| Situation | Required behaviour |
|---|---|
| New party at the same table | Provisioning layer obtains a fresh token for the new visit. Never reuse the old party's URL. |
| Offer is redeemed or cancelled | Device returns to neutral content; an optional lamp clears its reward signal. |
| Network or server unavailable | Show neutral/help content and stop promoting stale offers. No offline redemption. |
| Reboot or power loss | Kiosk starts automatically and retrieves current state rather than restoring a stale reward. |
| Device moved to another table | Authorised staff reassigns it. Audit the change; do not trust a table number entered by a guest. |
| Device lost or compromised | Revoke device access and current tokens; rotate only affected credentials where possible. |

## What the current browser screen already does

It polls every three seconds, refreshes its countdown every second, and clears offer data when the API reports a closed or invalid session. After repeated network failures lasting more than 15 seconds, it clears the stale content and shows a connection message. Staff redemption still checks server state regardless of the screen.

## What to add for a managed fleet

• A device registry with table assignment, revocation, heartbeat and last-seen health. These are proposed extensions, not existing database tables.

• Per-device credentials and narrow permissions, plus secure automatic visit-token handover. No service-role database keys on hardware.

• Watchdog/restart, managed updates, credential rotation and power-loss testing. For LED cues, enforce expiry locally even if a clear command is missed.

• A gateway that makes outbound authenticated connections, with constrained local access to controllers. Firmware capabilities determine the local protocol.

**For 20 browser displays polling every 3 seconds, plan for about 6.7 display requests per second, before staff traffic. This is a traffic estimate, not a tested capacity limit.**

Device clocks, guest-facing brightness, network isolation and physical safety must be tested on the selected hardware. TablePulse has not yet been exercised against real tablets, lamps or matrix controllers.

09 / INTEGRATION CONTRACT

# How it connects to the POS and n8n

## Inbound orders: implemented interface, mock provider

A vendor PosAdapter verifies the vendor message and normalizes it into a canonical order: externalId, sessionId, menuItemId/quantity lines, optional verified customerRef, and a discount flag. The signed request goes to POST /api/pos/events or its alias /api/offers/evaluate.

The signature is HMAC-SHA256 over timestamp + "." + the exact raw JSON body. Timestamps have a five-minute window. Identical retries with the same external ID are idempotent; changed payloads using that ID are rejected.

## What a real POS adapter still needs

• Map vendor menu IDs and tables/bills to TablePulse IDs; establish stable visit boundaries and verified customer references.

• Translate every discount type into a reliable stacking signal. Propagate voids, cancellations, table moves, bill closures and stock changes.

• Implement POS write-back or a documented staff entry workflow for redeemed purchases, with retries and reconciliation. Current redemption records a TablePulse bill line; it does not update an external POS.

• Retain vendor signature checks and credentials at the trusted boundary. Test duplicate, delayed and out-of-order events against the actual POS.

## n8n integration points

| Workflow | Connection |
|---|---|
| POS event routing | Receive vendor event -> normalize/sign -> call TablePulse order endpoint. Direct vendor integration is also possible. |
| Daily manager summary | Scheduled signed GET /api/integrations/summary?since=... -> your email or WhatsApp account. |
| Low stock and suspicious activity | Read summary fields; filter and deduplicate alerts before delivery. |
| Customer opt-out | Signed POST /api/integrations/optout -> disable opted-in preferences and return entitlement. |

**n8n orchestrates connections and notifications. PostgreSQL remains the source of truth and TablePulse remains the decision engine.**

No messaging workflow or external account is currently connected. Return-entitlement issuance, POS refunds and correction/reconciliation are integration work beyond the current canonical order contract. Payments stay in the existing POS.

10 / READINESS, MEASUREMENT AND REFERENCES

# From demo to a venue pilot

| Stage | Deliverable |
|---|---|
| 1. Confirm hardware | Select one browser screen or controller; validate power, mounting, Wi-Fi and readability. |
| 2. Deploy the service | Host Next.js on a Node-capable platform, create Supabase, apply migration/seed, set secrets and provision team users. Use HTTPS; turn demo mode off. |
| 3. Connect real operations | Implement and test the POS adapter, customer reference mapping and redemption write-back/manual process. |
| 4. Test two physical tables | Exercise new visits, cancellation, expiry, disconnect, reboot, redemption replay and staff safety blocks. Add fleet provisioning if unattended. |
| 5. Measure a controlled pilot | Compare completed visits with suitable controls. Track profit and guest/staff response before increasing allocations. |

## Costs and success criteria

Budget for per-table screens/controllers, mounts and power, hosting, Supabase, optional device management/gateway, POS integration work and messaging. Prices depend on providers and devices; no purchase quote is included. Measure incremental contribution after operating costs, conversion, discounts, stock availability, repeated failures and device uptime.

Existing verification includes 17 automated tests, HTTP workflow/security checks, TypeScript validation and a production build. Hosted Supabase Auth/Realtime, the real POS and physical devices still require integration testing. A live pilot also needs monitoring, backups, request rate limits and a data-retention policy.

## Code map

**lib/engine.ts**: eligibility/scoring · **lib/server/commands.ts**: mutations/redemption · **lib/server/store.ts**: demo/Supabase persistence · **lib/pos/adapter.ts**: POS contract · **components/public-screen.tsx**: display · **supabase/migrations/202610070001_tablepulse.sql**: schema, constraints, RLS and transactions · **README.md**: setup and deployment.

## Sources and further reading

Implementation descriptions are based on the TablePulse source and README in this workspace. Hardware patterns are proposed designs. Official external references checked 7 October 2026:

[1] Supabase - Row Level Security

[2] Raspberry Pi - Browser kiosk mode

[3] WLED - JSON API for light control

[4] Espressif - ESP-MQTT documentation

**For a first pilot: browser screens + Next.js + Supabase + one tested POS integration. Add a device gateway when the chosen hardware requires it. An AI agent is unnecessary for V1.**